پیچیدگی زمانی الگوریتم ها

هدف : استفاده از ابزار و روش های جهت مقایسه کارایی الگوریتم ها

برای حل یک مسئله روش های متفاوتی داریم که برای یافتن بهترین آن باید هزینه آن را دید تا کم هزینه ترین راه حل انتخاب شود.

برای اجرای یک الگوریتم زمان اجرا و حافظه مصرفی به عنوان هزینه در نظر گرفته می شود.

پس الگوریتمی که سریع تر اجرا و حافظه کمتری مصرف کند کارایی بیشتری دارد.

معمولا دو فاکتور فوق رابطه عکس با هم دارند به این معنی است که برای نوشتن الگوریتم سریع تر به حافظه بیشتری نیاز داریم و برعکس

با توجه به اهمیت بیشتر زمان اجرا کارایی یک الگوریتم با محاسبه زمان اجرای آن ارزیابی می شود.

منظور از محاسبه زمان اجرای یک الگوریتم تعداد ثانیه های سپری شده در طول اجرایش نیست بلکه برای دسترسی به یک معیار مشترک مستقل از سخت و نرم افزار تعداد دستوراتی که یک الگوریتم باید اجرا نماید را محاسبه کرده و به آن پیچیدگی زمانی الگوریتم در چند حالت ها می گیم

و با نماد $T(n)$ نمایش می دهیم.

پیچیدگی زمانی ← T

اندازه مسئله ← n

```
Func(int n, int[]arr)
{
   int i, max;
   max = arr[0]; ->  دستور ①
   For(i=1; i<arr.Lenth; i++){
       if (arr[i]>max)
           max = arr[i]
       }
     print(max); -> دستور ①
   }
```

حلقه For: ۳×(n-1) دستور

$\boxed{۳\times(n-1)+۲}$ => تعداد دستورهای اصلی برنامه

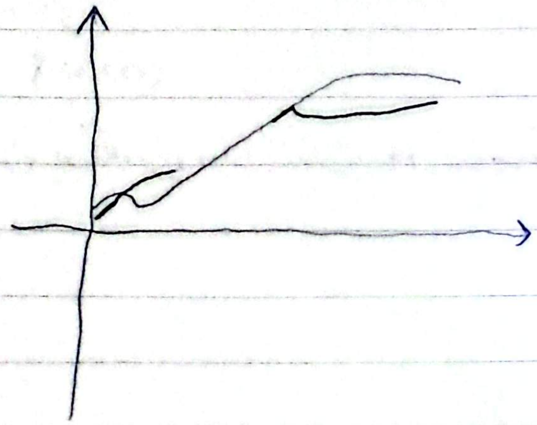
با استفاده از چند نماد O (بزرگ)، o (کوچک)، [illegible] و ~~Θ~~، پیچیدگی زمانی به دست آمده برای یک الگوریتم را به یک مقدار مشخص تخمین می‌زنیم تا مقایسه الگوریتم‌ها آسان‌تر انجام شود.

تعریف: O، big: اگر ثابت n0 ، c موجود باشد به طوری که داشته باشیم

$f(n) \leq c \times g(n) \quad n \geq n_0$

آنگاه می‌گوییم تابع f از نماد O بزرگ تابع g است.

$F \in O(g)$



تمام توابع چند جمله‌ای $f(n)$ به صورت زیر

$$f(n) = a_k n^k + a_{k-1} n^{k-1} + \_\_\_ + a_2 n^2 + a n + a_0$$

$f(n) \in O(n^k)$ از مرتبه O بزرگ n^k هستند

$f(n) = 3n^2 + n - 10 \in O(n^2)$

$f(n) = \cancel{b}n^3 \log n - n^2 - n \log n + 1000 \in O(n^3 \underbrace{\log n})$

$f_n = \log n + 10 \in O(\log(n))$ لگاریتم

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < \_\_\_ <$

$O(2^n) < O(n^n)$ ___ رده الگوریتم

برای مقایسه زمانی دو عدد $T(n)$ داریم باید بزرگترین توان ها رو مقایسه کنیم

هرکدام که بزرگتر بود کندتر است.

مثال: می خواهیم دو الگوریتم برای جستجوی یک عدد در آرایه بنویسیم

۱- جستجوی خطی

۲- جستجوی ~~ج~~ دو دویی

ضمن نوشتن در الگوریتم فوق با زبان C++ پیچیدگی آن ها با هم مقایسه کنید

Subject:

Date:

```
          int x[]
int check( int 2){
 for(int i=0; i<x[n]; i++){
   if(2 == x[i]){
     cout<<"1";
   }
 }}
```

نسخه نوشته شده استاد

```
 int SegSearch(inn num,int[] arr)
{int i;
  for(i=0; i<arr.Lenth; i++){
   if(arr[i]==num){
     return i;
   }}
    return -1;}}
 main(){
     int num,R;
     int []arr= مقدار دهی
   R=SegSearch(num,arr);
   if(R==-1) cout<<"not found";
else cout<<"Founded";
```

در اینجا یعنی T(n) ها ۳ مورد داریم

B(n) => بهترین حالت

A(n) => میانگین

W(n) => بدترین

$B(n) = 1 \in O(1)$

$A(n) = \frac{n}{2} \in O(n)$

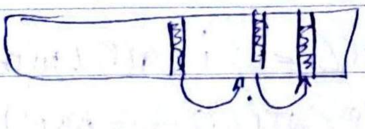
$W(n) = n \in O(n)$

Subject: Date:

جستجو دودویی

```
int binSearch(int num, int[]arr)
{
  int mid; int low=0; int high=arr.Lenth-1;
  while (low<=high)
  {
  mid=(low+high)/۲;
  if(arr[mid]==num) return mid;
  else if(arr[mid]<num) low=mid+1;
  else high=mid-1;
  }
return -1;
}
```

در این مدل همیشه مرکز در نظر می‌گیریم
اگر عدد کمتر بود نیمه سمت راست
را در نظر می‌گیریم و عدد مرکز
می‌گیریم و اگر بیشتر
بود نیمه سمت چپ در نظر می‌گیریم



تعداد تکرار حلقه	تعداد اعداد محدوده	
۱	$N/۲^{۰}$	۰
۱	$N/۲^{۱}$	۱
۱	$N/۲^{۲}$	۲
۱	↓	
۱		
۱	$n/۲^{k}$	k
k+1		

$۲^{k}=N$

$k=\log_{۲}^{N}$

$B(n)=1$

$W(n)=\log_{۲}^{n}+1 \in O(\log(n))$

پس تابع دودویی کوچک‌تر و سریع‌تر است و الگوریتم بهتری است.

AFAGH